

Security Audit

Indie.fun (Gaming)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Indie.fun team partnered with Hashlock to conduct a security audit of their campaign.rs and proxy_initialize.rs smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Indie.fun is an upcoming platform for indie game developers and enthusiasts, set to launch in January 2025. The project aims to provide a space for indie game creators to showcase their work, connect with players, and potentially monetize their creations.

Simply put, Indie.fun is a hub where indie game developers can submit their games, players can discover unique gaming experiences, and the indie gaming community can thrive through direct connections between creators and fans.

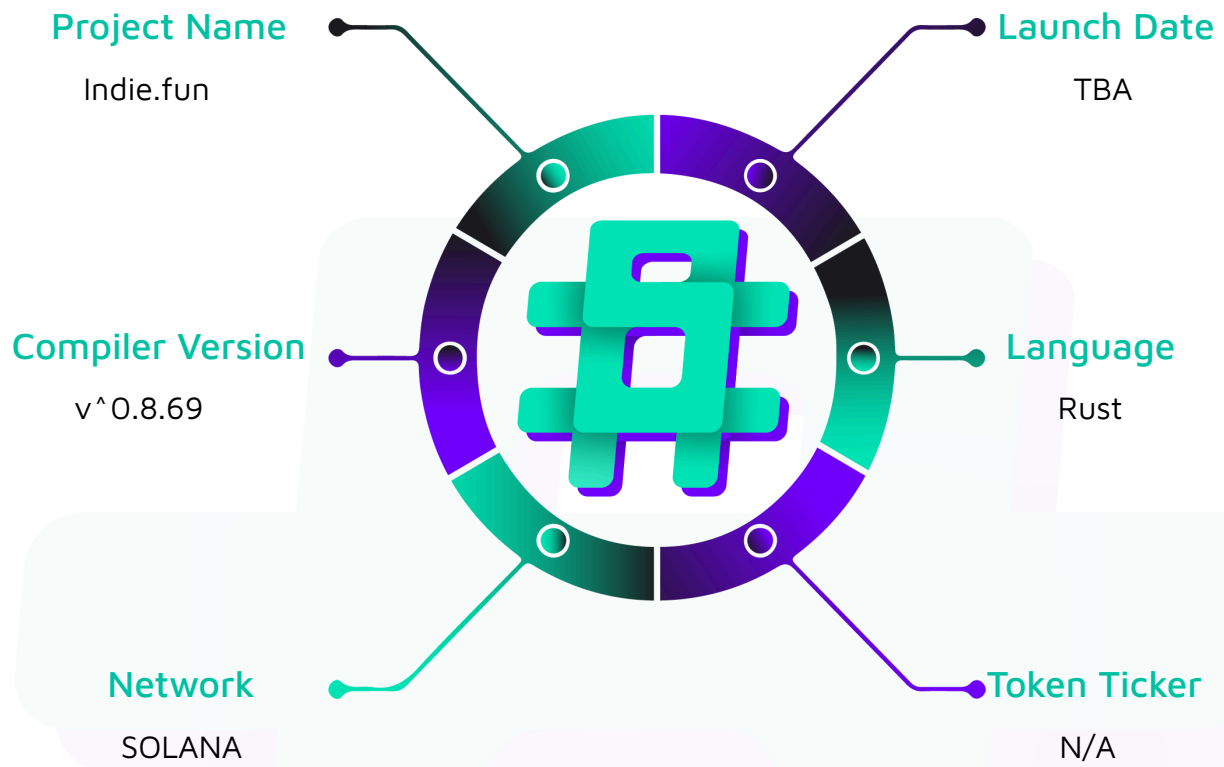
Project Name: Indie.fun

Compiler Version: v^0.8.69

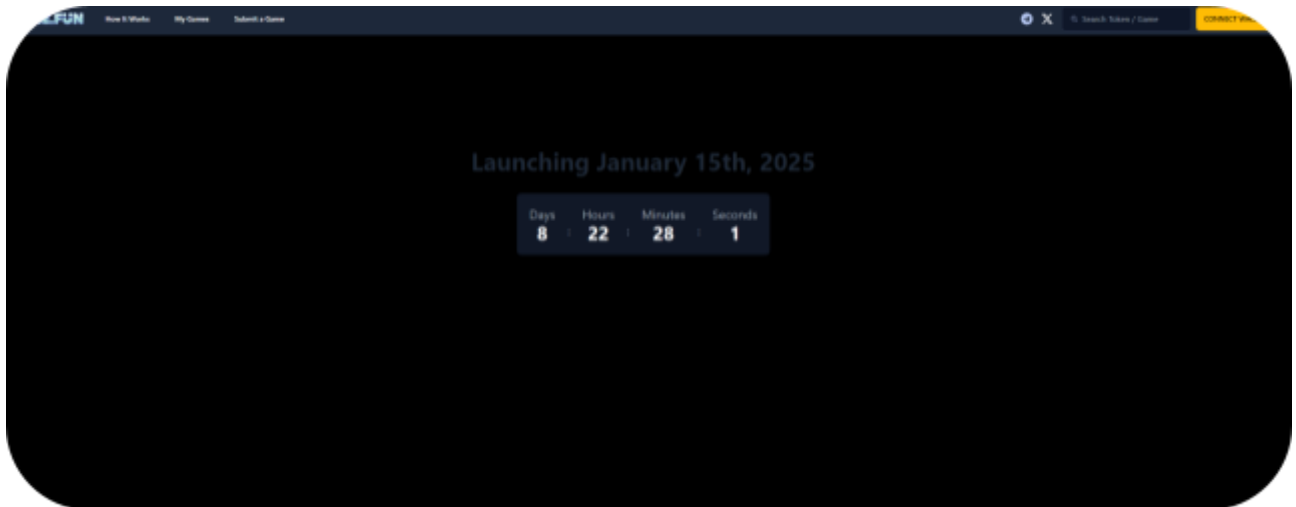
Website: <https://indie.fun/>

Logo:

INDIE.FUN

Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the rust code within the Indie.fun project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Indie.fun Protocol Smart Contracts
Platform	Rust
Audit Date	January, 2025
Contract 1	campaign.rs
Contract 1 MD5 Hash	c7762e9cfa54d45ac55c8e9aab33cfa3
Contract 2	proxy_initialize.rs
Contract 2 MD5 Hash	f50ad3835b2b0210ec1c9c12336d3022
GitHub Commit Hash	8c445ee7b0ec86ad24dd58b4df7156462cdec7d4

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. We initially identified some significant vulnerabilities that have since been addressed.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
campaign.rs - Allows users to: - Create a token and start a crowdfunding campaign (to launch a game later on using the token as in-game currency).	Contract achieves this functionality.
proxy_initialize.rs - Initialize a liquidity pool using the tokens that were backed during the campaign	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Indie.fun project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Indie.fun project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] proxy_initialize.rs - No constraint is provided on the admin account

Description

Currently, there is an admin account that's provided among other accounts in the proxy_initialize.rs contract. The problem is that there are no any constraints for this account provided, basically allowing any signer to be represented as an admin account.

Vulnerability Details

At the moment, any signer can be imitated as admin which is not an expected behavior as an admin account has to be compared with an admin wallet provided in the constants.rs :

```
#[account(mut)]  
  
pub admin: Signer<'info>,
```

It's similar to another instruction case where such a constraint is put that the admin is indeed a trusted wallet. However, in this case, it's needed to be ensured that the admin is actually a trusted address as this account is later used when instruction is performed:

```
CloseAccount {  
  
    account: ctx.accounts.campaign_lp_token.to_account_info(),  
    destination: if ctx.accounts.campaign.lock_lp {  
        ctx.accounts.treasury.to_account_info()  
    } else {  
        ctx.accounts.admin.to_account_info()  
    }  
}
```

```
    },  
  
    authority: ctx.accounts.campaign_vault.to_account_info(),  
  
  },
```

So even the money can be sent to this account as a destination when closing them.

Impact

Anybody can call the function on behalf of the admin.

Recommendation

Provide the constraint that would compare the admin account with the `ADMIN_WALLET` pubkey from the `constants.rs`.

Status

Resolved

Medium

[M-01] constants.rs - Duplicate values for the same parameters in the constants

Description

In the current implementation, there are several parameters that are duplicated or are not properly named and not used correctly.

Vulnerability Details

constants.rs file has the following values that describe different parameters used when creating campaigns. There are such parameters as MIN_FUNDRAISING_GOAL, MIN_LIQUIDITY_POOL_AMOUNT, POOL_CREATION_FEE and that are expected to have the same meaning but that have totally different values despite the same naming. On the other hand, they may be incorrectly named and one of the parameters has to be "MAX" and not "MIN" as their values significantly differ from each other. But, in that case, these max values are not checked when creating campaigns at all.

```
#[cfg(feature = "devnet")]

pub const MIN_LIQUIDITY_POOL_AMOUNT: u64 = 5 * LAMPORTS_PER_SOL;

#[cfg(not(feature = "devnet"))]

pub const MIN_LIQUIDITY_POOL_AMOUNT: u64 = (6 * LAMPORTS_PER_SOL) / 10; // 0.6 SOL

#[cfg(feature = "devnet")]

pub const MIN_FUNDRAISING_GOAL: u64 = 15 * LAMPORTS_PER_SOL;

#[cfg(not(feature = "devnet"))]

pub const MIN_FUNDRAISING_GOAL: u64 = 2 * LAMPORTS_PER_SOL;

#[cfg(feature = "devnet")]
```

```
pub const POOL_CREATION_FEE: u64 = (LAMPORTS_PER_SOL * 1) +
POOL_ACCOUNT_CREATION_FEE; // 1 SOL + account creation fee

#[cfg(not(feature = "devnet"))]

pub const POOL_CREATION_FEE: u64 = (LAMPORTS_PER_SOL * 15) / 100 +
POOL_ACCOUNT_CREATION_FEE; // 0.15 SOL + account creation fee
```

All the parameters have the same naming here even though they have distinct underlying values.

Impact

Improper usage of the constants / not using intended values for the parameters.

Recommendation

Consider removing one of the parameters not to have 2 of the same ones and if one of the parameters is expected to be the “MAX” value and not “MIN” - check whether they’re enforced in the `campaign.rs` or not.

Note

Indie.fun team informed Hashlock that the use of different values for `MIN_LIQUIDITY_POOL_AMOUNT`, `MIN_FUNDRAISING_GOAL`, and `POOL_CREATION_FEE` during testing (devnet) is intentional and serves to simplify the testing process. Raydium charges different `POOL_CREATION_FEE` rates for devnet and mainnet, which is why they need to apply distinct constant values for each network environment to ensure accurate testing.

Status

Acknowledged

[M-02] campaign.rs#back_campaign() - The total number of the backers is not increased when funds are deposited into the campaign

Description

The state is not updated for the total backers of the campaign when funds are deposited there.

Vulnerability Details

The current functionality inside of the campaign.rs contract does not update the number of the total backers upon campaign creation. When there is a backer that deposits the funds into the campaign vault, the only state variable that's updated is the number of the total_backed funds. However, there is also a total_backers parameter that's not updated at all.

```
campaign.total_backers = 0;
```

Impact

The state is not updated correctly for the campaign info.

Recommendation

Increase the number of total backers when the funds are deposited into the campaign vault.

Status

Resolved

[M-03] proxy_initialize.rs#initialize_liquidity_pool() - Tokens are burnt instead of locking them up as the name of the parameter suggests

Description

Currently, if the lock_lp parameter is set to true, the tokens are burnt. But this is not correct as burning the tokens is not the same thing as locking them up.

Vulnerability Details

There is a `lock_lp` parameter that's used to decide whether to lock the tokens or to transfer them to the creator's account:

```
if ctx.accounts.campaign.lock_lp {

    // Burn LP tokens

    anchor_spl::token::burn(

        CpiContext::new_with_signer(

            ctx.accounts.token_program.to_account_info(),

            anchor_spl::token::Burn {

                mint: ctx.accounts.lp_mint.to_account_info(),

                from: ctx.accounts.campaign_lp_token.to_account_info(),

                authority:

ctx.accounts.campaign_vault.to_account_info(),

            },

            &signer_seeds,

        ),

        campaign_lp_token_account.amount,

    )?;

} else {

    // Transfer LP tokens to campaign creator

    handle_lp_token_transfer(

        &ctx,

        &signer_seeds,

        campaign_lp_token_account.amount,

    )?;
```

```
}UncheckedAccount<'info>,
```

As you can see here, if the parameter's value is set to false, the tokens are basically burnt but the parameter is called lock_lp meaning the tokens have to be locked that is sent to some predetermined escrow address.

Impact

The tokens are unintentionally burnt instead of locking them up.

Recommendation

Lock the tokens and not burn them.

Status

Resolved

Low

[L-01] `proxy_initialize.rs` - Unnecessary double checking for the first token of the pool

Description

When initializing a liquidity pool, there is a check that makes sure whether the first token is WSOL or not. This is done repeatedly which is not correct and the first check can be used for all the subsequent logic.

```
let is_sol_token_first = ctx.accounts.token_0_mint.key() == WSOL_MINT;

// Handle WSOL transfers and sync

let wsol_lp_amount = {

    let is_sol_token_first = ctx.accounts.token_0_mint.key() == WSOL_MINT;
```

Recommendation

Remove the second check for the first token of the pool.

Status

Resolved

[L-02] `campaign.rs#create_campaign()`- Creation of campaigns is not permissionless deviating the spec

Description

According to the documentation provided by the protocol, any entity has to be able to create a new campaign for a game.

Vulnerability Details

The current functionality of the protocol is expected to have the functionality for permissionless crowdfunding campaign creation for any token (currently existing one or a completely new one). However, the contract's implementation at the moment deviates from the intended specification allowing only admin to create the corresponding campaigns.

Proof of Concept

Currently, the following accounts are provided to the `create_campaign()` function:

```
pub struct CreateCampaign<'info> {  
  
    #[account(mut)]  
  
    pub creator: Signer<'info>,  
  
    // admin signature will help to ensure campaign  
  
    // creator of a game actually owns the game.  
  
    #[account(constraint = admin.key() == ADMIN_WALLET)]  
  
    pub admin: Signer<'info>,  
}
```

As you can see here, both the admin account and the creator account are the same but there is a constraint for the admin key to be the `ADMIN_WALLET` value disallowing anybody else to create campaigns.

Impact

Users are not able to create permissionless crowdfunding campaigns.

Recommendation

Remove the admin account as a signer from the list of accounts.

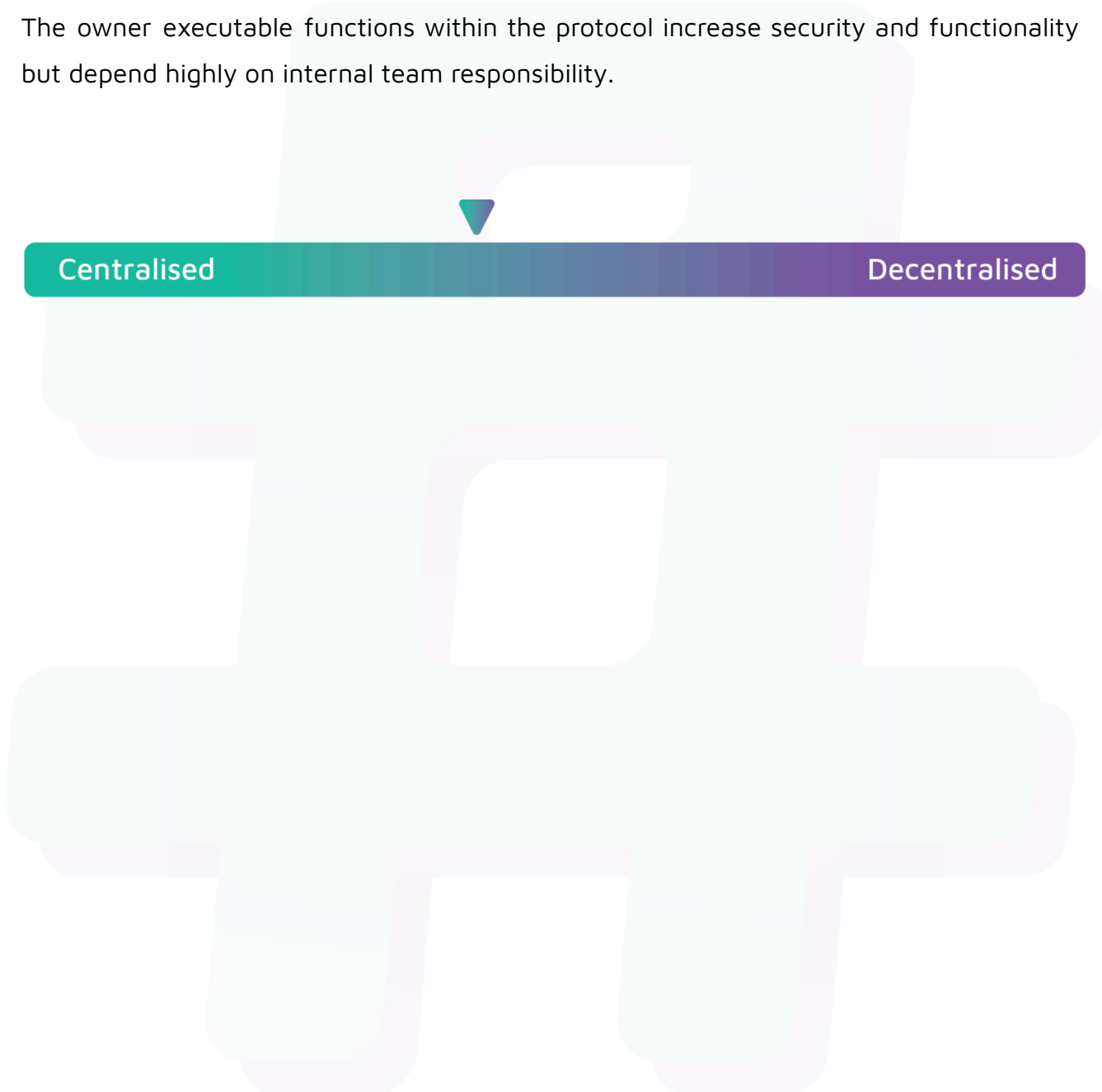
Status

Resolved

Centralisation

The Indie.fun project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Indie.fun project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.